PRACTICAL MILESTONE

Lab 6 of 8

Reinforcement learning

Learn a policy by interacting, then save it

Coverage. Chapter 20.
Goal. Train tabular Q-learning, evaluate the greedy policy, visualize values/actions, and save the Q-table.

Setup.

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

PROJECT_ROOT = Path.cwd().parent
MODELS = PROJECT_ROOT / "models"
MODELS.mkdir(exist_ok=True)
```

Part A — Tabular Q-learning in a 4×4 GridWorld.
Use a 4×4 grid: start top-left, goal bottom-right, and a trap. Ordinary moves cost -0.04, the goal pays $+1$, and the trap pays -1 and terminates. Walls leave position unchanged. `step` returns state, reward, and termination.

```
rng = np.random.default_rng(42)

ROWS, COLS = 4, 4
START, GOAL, TRAP = (0, 0), (3, 3), (1, 3)
ACTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)]        # up, down, left, right
ACTION_NAMES = ["up", "down", "left", "right"]
ARROWS = {"up": (0, 1), "down": (0, -1), "left": (-1, 0), "right": (1, 0)}

def step(state, action):
    """Apply one action. Returns (next_state, reward, done)."""
    r, c = state
    dr, dc = ACTIONS[action]
    next_state = (min(max(r + dr, 0), ROWS - 1), min(max(c + dc, 0), COLS - 1))
    if next_state == GOAL:
        return next_state, 1.0, True
    if next_state == TRAP:
        return next_state, -1.0, True
    return next_state, -0.04, False
```

Initialize state–action Q-values to zero. Update toward reward plus discounted best next-state value, with step size α and discount γ. Decay exploration from $\varepsilon = 1$.

```
Q = np.zeros((ROWS, COLS, len(ACTIONS)))
alpha, gamma = 0.15, 0.95
epsilon, min_epsilon = 1.0, 0.05

def choose_action(state, explore=True):
    if explore and rng.random() < epsilon:
        return int(rng.integers(len(ACTIONS)))       # explore: random move
    return int(np.argmax(Q[state]))                  # exploit: best known move

training_returns = []
for episode in range(5000):
    state, episode_return = START, 0.0
    for _ in range(100):
        action = choose_action(state)
```

```
        next_state, reward, done = step(state, action)
        episode_return += reward
        target = reward if done else reward + gamma * np.max(Q[next_state])
        Q[state][action] += alpha * (target - Q[state][action])
        state = next_state
        if done:
            break
    training_returns.append(episode_return)
    epsilon = max(min_epsilon, epsilon * 0.9985)
```

Plot returns smoothed over 100 episodes as exploration decays.

```
window = 100
moving = np.convolve(training_returns, np.ones(window) / window, mode="valid")
plt.figure(figsize=(6, 3.6))
plt.plot(np.arange(window - 1, len(training_returns)), moving)
plt.axhline(0.8, linestyle="--", color="grey", label="best possible return (6 moves)")
plt.xlabel("Episode")
plt.ylabel("Mean return over last 100 episodes")
plt.title("Q-learning training curve")
plt.legend()
plt.tight_layout()
plt.show()
```

Evaluate separately with exploration off, choosing the largest Q-value in each cell.

```
successes, returns = 0, []
for _ in range(200):
    state, total = START, 0.0
    for _ in range(100):
        state, reward, done = step(state, choose_action(state, explore=False))
        total += reward
        if done:
            successes += int(state == GOAL)
            break
    returns.append(total)

print("success rate:", successes / 200)
print("mean return:", round(float(np.mean(returns)), 3))
print("learned action at start:", ACTION_NAMES[choose_action(START, explore=False)])
```

Inspect. Six moves yield $5(-0.04) + 1 = 0.80$. A 100% evaluated success rate at return 0.8 indicates the shortest route.

Part B — The Q-table is the model: save it and read the policy.
Save the Q-table; deployment chooses each state's highest-valued action without retraining.

```
q_path = MODELS / "gridworld_q_table.npy"
np.save(q_path, Q)

Q_loaded = np.load(q_path)
policy = np.full((ROWS, COLS), "", dtype=object)
for r in range(ROWS):
    for c in range(COLS):
        policy[r, c] = "GOAL" if (r, c) == GOAL else "TRAP" if (r, c) == TRAP \
            else ACTION_NAMES[int(np.argmax(Q_loaded[r, c]))]
print(policy)
```

Plot greedy actions as arrows over maximum Q-values.

```
def plot_policy(Q_table, title):
    values = Q_table.max(axis=2)
    plt.figure(figsize=(4.4, 4.2))
    plt.imshow(values, cmap="YlGn")
    plt.colorbar(label="value of the best action")
    for r in range(ROWS):
        for c in range(COLS):
            if (r, c) == GOAL:
                plt.text(c, r, "GOAL", ha="center", va="center", fontweight="bold")
            elif (r, c) == TRAP:
                plt.text(c, r, "TRAP", ha="center", va="center", color="red", fontweight="bold")
```

```
            else:
                dx, dy = ARROWS[ACTION_NAMES[int(np.argmax(Q_table[r, c]))]]
                plt.arrow(c, r, 0.3 * dx, -0.3 * dy, head_width=0.15, color="black")
    plt.xticks(range(COLS))
    plt.yticks(range(ROWS))
    plt.title(title)
    plt.tight_layout()
    plt.show()

plot_policy(Q_loaded, "Greedy policy and state values")
```

Inspect. Trace routes toward the goal and around the trap; compare neighboring state values.

Try it yourself

Change the step cost, trap, exploration schedule, or discount. Predict the policy change, retrain, and check with `plot_policy`.

Verification checklist

- The greedy policy reaches the goal in six moves, for a return of 0.80.
- The evaluated success rate is reported over episodes run without exploration.
- `models/gridworld_q_table.npy` is saved, and the policy is read from the reloaded table.
- The printed policy routes towards the goal and around the trap.

Check your understanding

1. What are the state, action, reward, and transition in a reinforcement-learning formulation?
2. What two pieces of information appear in the tabular Q-learning backup value?
3. Why should the exploratory training policy be evaluated separately from the final greedy policy?